

Dynamic Memory Allocation in C (malloc, calloc, realloc, free)

📌 Definition to Remember

Dynamic Memory Allocation (DMA) allows programs to request, resize, and release memory from the Heap segment at runtime using library functions from `stdlib.h`.

1. Concept Overview & Fundamentals 🧠

Dynamic Memory Allocation in C (`malloc`, `calloc`, `realloc`, `free`) forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

- Code is compiled directly into native machine instructions without virtual machine overhead.
- Memory is strictly partitioned into **Code/Text Segment**, **Data Segment (Initialized/BSS)**, **Stack**, and **Heap**.
- Direct pointer and register manipulation provides deterministic performance critical for operating systems, embedded hardware, and database kernels.

2. Technical Architecture & Memory Mechanics ⚙️

Memory Layout & Storage Organization

```
+-----+
|                STACK SEGMENT                |
| - Local variables, function call stack frames, parameters |
| - High memory growing downwards towards Heap |
+-----+
|                |
|                v                ^                |
|                |                |                |
+-----+
|                HEAP SEGMENT                |
| - Dynamic memory allocations via malloc(), calloc() |
| - Low memory growing upwards towards Stack |
+-----+
|                INITIALIZED DATA SEGMENT        |
| - Global and static variables with non-zero initial values |
+-----+
|                BSS SEGMENT (UNINITIALIZED)        |
| - Global and static variables initialized to zero |
+-----+
|                CODE / TEXT SEGMENT                |
| - Read-only executable binary instructions |
+-----+
```

Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers dynamic scalability	Stack: O(1) push/pop; Heap: O(1) alloc
Type Checking	Static Typing	Compiler verifies types during compilation	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memory address	Direct memory dereference (O(1))
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops, and call stacks	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: Dynamic Memory Allocation\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.
- **Embedded & IoT Devices:** Microcontrollers with constrained RAM (8KB-64KB) require C's predictable footprint.
- **Database Engine Kernels:** High-throughput storage engines (PostgreSQL, SQLite, MySQL InnoDB) rely on low-level buffer pool managers written in C.
- **Game Engine Foundations:** Graphics rasterization, physics engines, and GPU interfaces utilize C/C++ memory arrays for zero-overhead computation.

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: calloc sentence

Problem Description & Objective:

Implementation of calloc sentence

C Code Implementation:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    /*
     * Logic:
     * - Goal: Allocates dynamic character array using calloc, reads a sentence, and releases the memory.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int size;
    printf("Enter maximum sentence length: ");
    scanf("%d", &size);

    char *str = (char *)calloc(size, sizeof(char));

    if (str == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    getchar(); // clear newline

    printf("Enter sentence: ");
    fgets(str, size, stdin);

    printf("You entered: %s", str);

    free(str);
    return 0;
}
```

Problem 2: car malloc

Problem Description & Objective:

Implementation of car malloc

C Code Implementation:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Car {
    char brand[50];
    int year;
    float price;
};

int main() {
    /*
     * Logic:
     * - Goal: Allocates dynamic memory for a car structure using malloc, initializes fields, and frees
memory.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    struct Car *c;

    // allocate memory
    c = (struct Car *)malloc(sizeof(struct Car));

    if (c == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    // assign values
    strcpy(c->brand, "Toyota");
    c->year = 2022;
    c->price = 1500000.50;

    // print
    printf("Brand: %s\nYear: %d\nPrice: %.2f\n", c->brand, c->year, c->price);

    // free memory
    free(c);

    return 0;
}

```

Problem 3: float array malloc

Problem Description & Objective:

Implementation of float array malloc

C Code Implementation:

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    /*
     * Logic:
     * - Goal: Dynamically allocates memory for an array of floats, populates elements, and calculates
     sum/average.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    float *arr = (float *)malloc(n * sizeof(float));

    if (arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    // input
    for (int i = 0; i < n; i++) {
        printf("Enter value %d: ", i + 1);
        scanf("%f", &arr[i]);
    }

    // output
    printf("Values:\n");
    for (int i = 0; i < n; i++) {
        printf("%.2f ", arr[i]);
    }

    free(arr);
    return 0;
}

```

Problem 4: point struct malloc

Problem Description & Objective:

Implementation of point struct malloc

C Code Implementation:

```

#include <stdio.h>
#include <stdlib.h>

struct Point {
    int x;
    int y;
};

int main() {
    /*
     * Logic:
     * - Goal: Dynamically allocates memory for a point structure, stores coordinate values, and releases
memory.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    struct Point *p;

    p = (struct Point *)malloc(sizeof(struct Point));

    if (p == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    // assign values
    p->x = 10;
    p->y = 20;

    printf("Point: (%d, %d)\n", p->x, p->y);

    free(p);

    return 0;
}

```

Problem 5: realloc shrink

Problem Description & Objective:

Implementation of realloc shrink

C Code Implementation:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    /*
     * Logic:
     * - Goal: Demonstrates how to resize dynamically allocated memory blocks using realloc.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int *arr = (int *)calloc(n, sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    // random numbers
    srand(time(0));
    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 100;
    }

    printf("Original array:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    // shrink size
    int newSize = n / 2;
    arr = (int *)realloc(arr, newSize * sizeof(int));

    printf("\n\nAfter shrinking:\n");
    for (int i = 0; i < newSize; i++) {
        printf("%d ", arr[i]);
    }

    free(arr);
    return 0;
}
```

6. Exam Scoring Points & Memory Revision

★ Must-Write Points

1. `malloc(size)`: Allocates raw uninitialized memory bytes; returns `void*` (or `NULL` on failure).
2. `calloc(n, size)`: Allocates and zero-initializes contiguous memory blocks.
3. `realloc(ptr, new_size)`: Resizes previously allocated heap block, preserving data.
4. `free(ptr)`: Deallocates heap memory back to the operating system.
5. Memory Leak: Failing to call `free()` causes memory exhaustion; Dangling Pointer: Accessing memory after `free()`.

⚡ Quick Recall

`malloc / calloc (Heap) -> NULL Check -> Use Buffer -> realloc -> free(ptr) -> ptr = NULL`